



NOSQL

The database revolution

1

Introduction to
NoSQL

2

NoSQL Models

3

Comparisons

4

Document
based model

5

Example

ONCE UPON A TIME

Information Retrieval

P. BAXENDALE, Editor

A Relational Model of Data for Large Shared Data Banks

E. F. CODD

IBM Research Laboratory, San Jose, California

Future users of large data banks must be protected from having to know how the data is organized in the machine (the internal representation). A prompting service which supplies such information is not a satisfactory solution. Activities of users at terminals and most application programs should remain unaffected when the internal representation of data is changed and even when some aspects of the external representation are changed. Changes in data representation will often be needed as a result of changes in query, update, and report traffic and natural growth in the types of stored information.

and terminal activities from growth in data types and changes in data representation—and certain kinds of *data inconsistency* which are expected to become troublesome even in nondeductive systems.

Volume 13 / Number 6 / June, 1970

The relational view (or model) of data described in Section 1 appears to be superior in several respects to the graph or network model [3, 4] presently in vogue for non-inferential systems. It provides a means of describing data with its natural structure only—that is, without superimposing any additional structure for machine representation purposes. Accordingly, it provides a basis for a high level data language which will yield maximal independence between programs on the one hand and machine representation and organization of data on the other.

A further advantage of the relational view is that it forms a sound basis for treating derivability, redundancy, and other data management issues.

Volume 13 / Number 6 / June, 1970

tions (see remarks in Section 2 on the “connection trap”).

Finally, the relational view permits a clearer evaluation of the scope and logical limitations of present formatted

parts might be stored in ascending order by part serial number. Such systems normally permit application programs to assume that the order of presentation of records from such a file is identical to (or is a subordering of) the

Communications of the ACM 377

POSITIVE ASPECTS OF RELATIONAL MODEL

- Relational model is very strict
 - Closed world assumption
 - Minimization principle
 - RDBMS (the software able to manage the relational model)
 - More than 35 years of R&D (security, optimization, standardization)
 - ACID properties
 - Very well known
 - A large amount of data is still stored in DBMS
 - Porting data is a nightmare!
 - For a large number of tasks it is still the best option
- ACID properties
 - Atomic
 - Consistence
 - Isolation
 - Durability

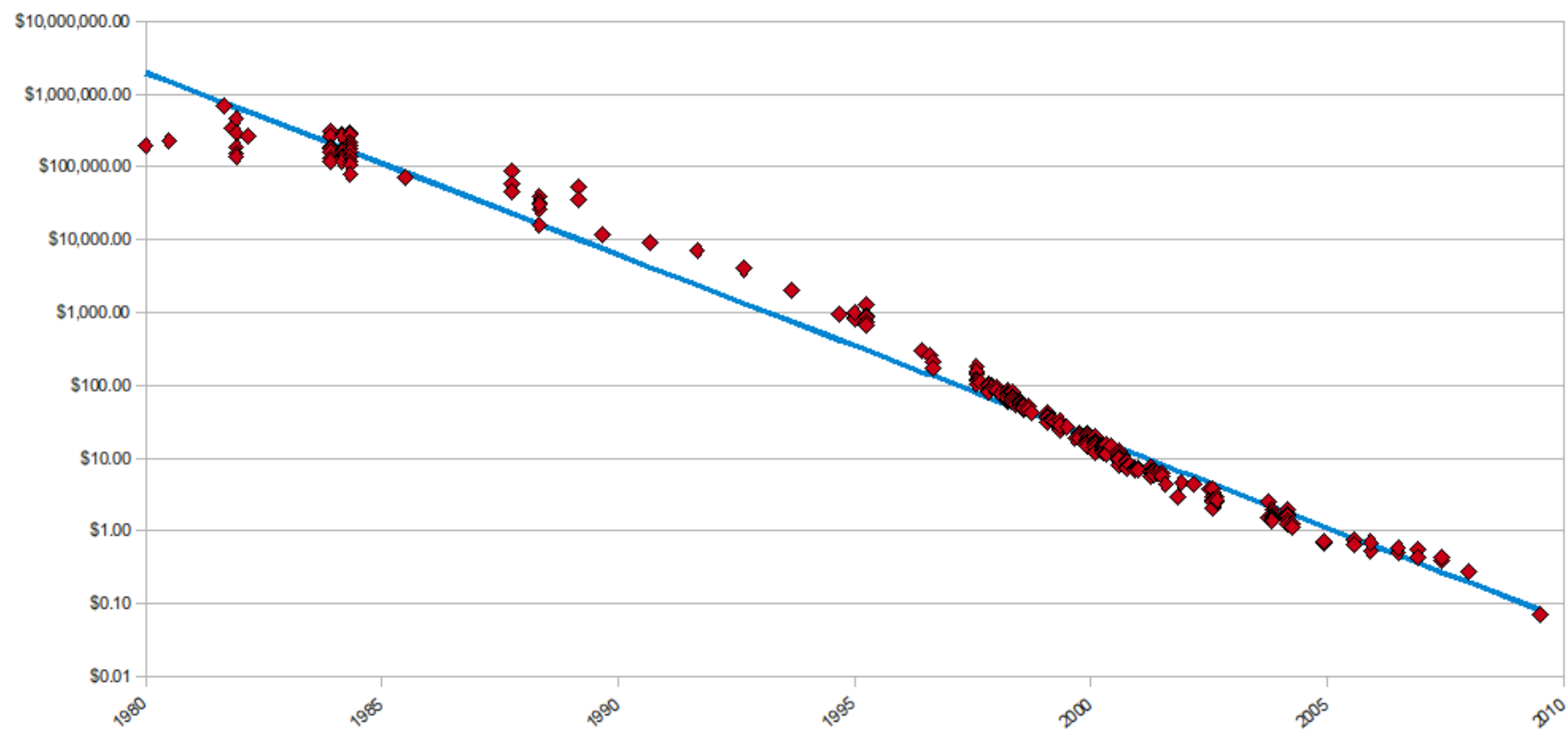
LIMITATION OF RELATIONAL MODEL

- Relational model is very stricted
 - Closed world assumption
 - Minimization principle
 - One attribute $\leftrightarrow$ one value
 - Not compatible with the modern programming language
 - It is not able to support cyclical data relationships (see later)
- RDBMS (the software able to manage the relational model)
 - Difficult to modify the tables
 - Not scalable

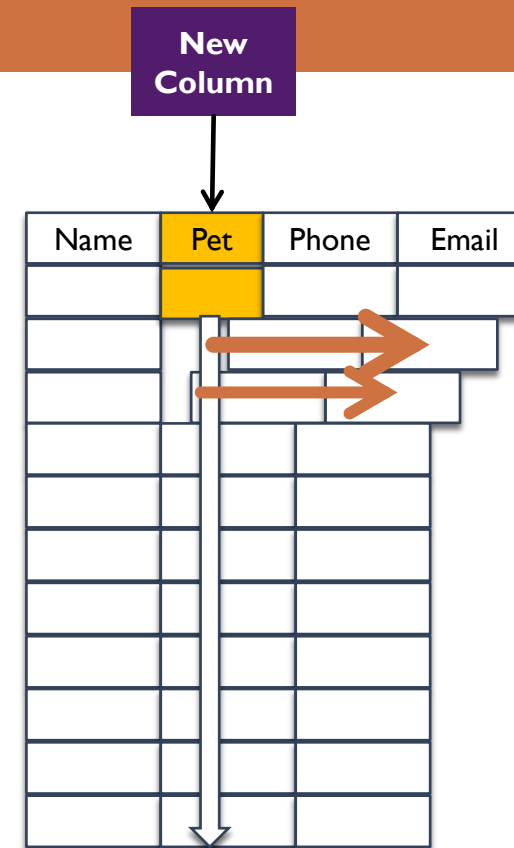
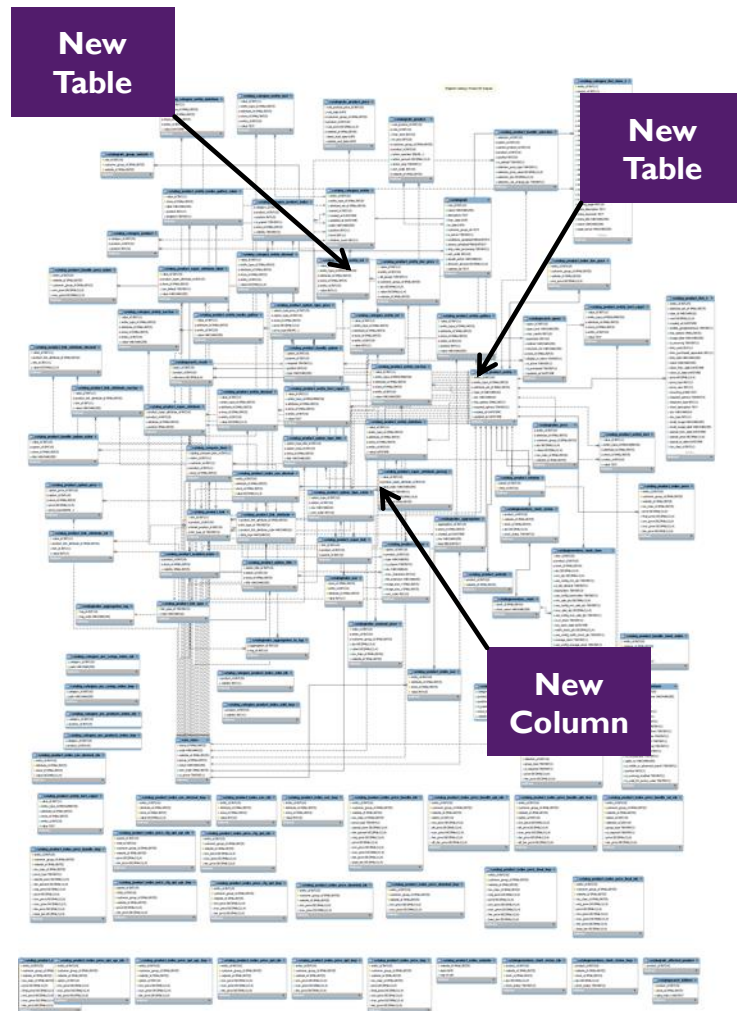
STORAGE!



Hard Drive Cost per Gigabyte
1980 - 2009



HARD TO MODIFY EXISTING DATA



3 months later...

PERFORMANCE

- How many seconds are needed for execute a given query?
 - It depends on many factors
 - number of rows
 - Type of query
 - Algorithm selection
 - Implementation of algorithms and data structure

PERFORMANCE

What is the Rossi Maria list of passed exams?

- (in general) The slowest type of query in the relational model is the one that involve more than one table
- The laziest algorithm (worst case):
- For each row in student
 - find the one with surname=Rossi and Name=Maria →select the badge number
 - For each row in Exam
 - find the one where Student=badgeNumber → select Course and grade
 - For each row in Course
 - find the one where code=course →select title

Student

Badge	Surname	Name	Date of birth
-------	---------	------	---------------

Exam

Student	Grade	Course
---------	-------	--------

Course

Code	Title	Professor
------	-------	-----------

PERFORMANCE

- RDBMSs are able to scale up easily, less scale out

- Scale up

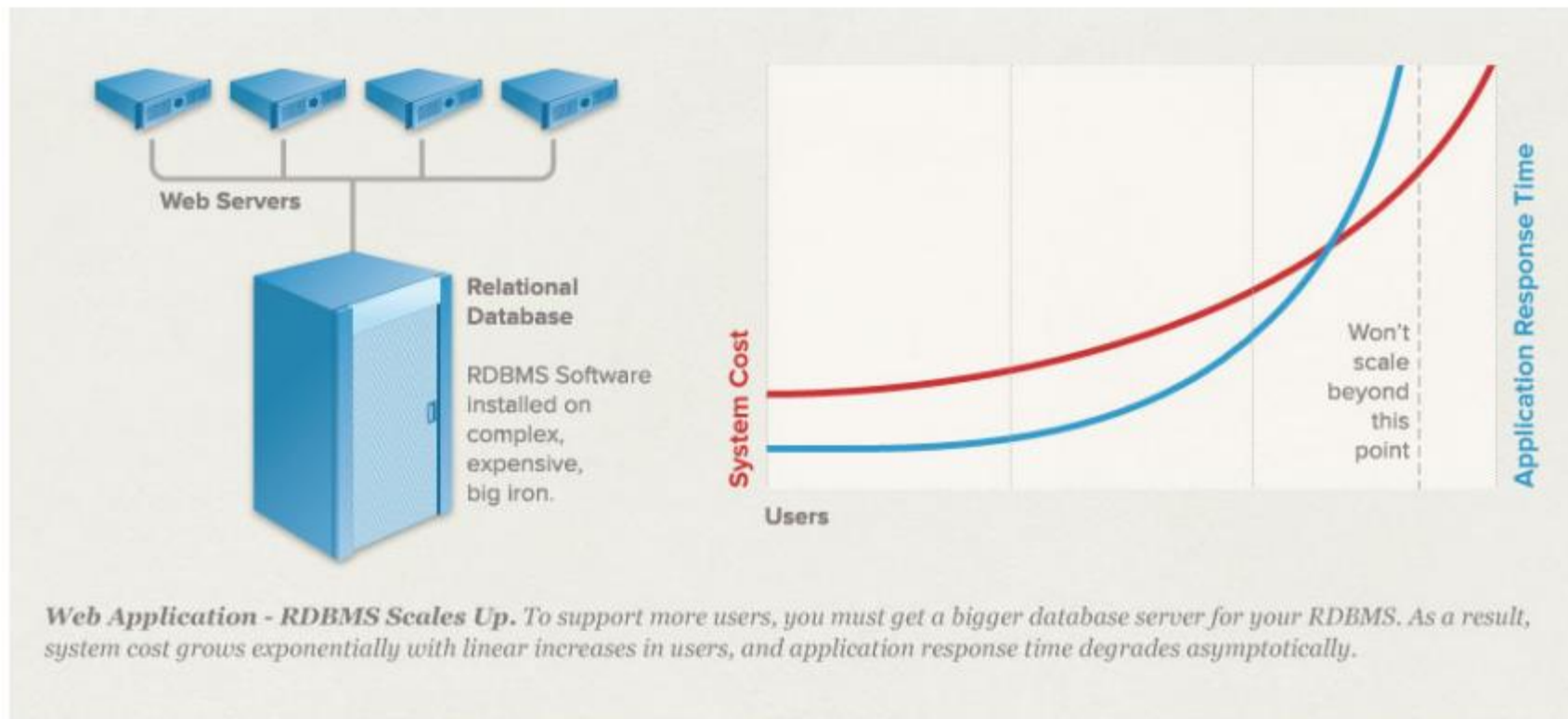


- Scale out



PERFORMANCE

- You cannot add servers forever



THE HISTORY OF NOSQL MOVEMENT

- Carlo Strozzi used the term NoSQL in 1998 to name his lightweight, open-source relational database that did not expose the standard SQL interface.
- Graph database Neo4j is started in 2000.
- Google BigTable is started in 2004. Paper published in 2006.
- CouchDB is started in 2005.
- The Amazon Dynamo research paper is published in 2007.
- The document database MongoDB is started in 2007 as a part of a open source cloud computing stack and first standalone release in 2009.
- Facebook opens the sources of the Cassandra project in 2008.
- Project Voldemort started in 2008.
- The term NoSQL was reintroduced in early 2009.

NOSQL

- Not only SQL
- It is a set of data models and related software
- Schema free (or schemaless)
- CAP theorem
- Base principle
 - Basic Available, Soft state, Eventually consistency

SCHEMA FREE

- In the relational model usually
- First define the model (the set of attributes that describe the data and their relationships)
 - Then the data is entered
- If you need to add a new attribute (or modify an existing one)
 - First modify the model, then (if possible) modify the data
- In most NoSQL models there is no strict model; is based on the data you enter (see below)
- All NoSQL models assume the open world assumption

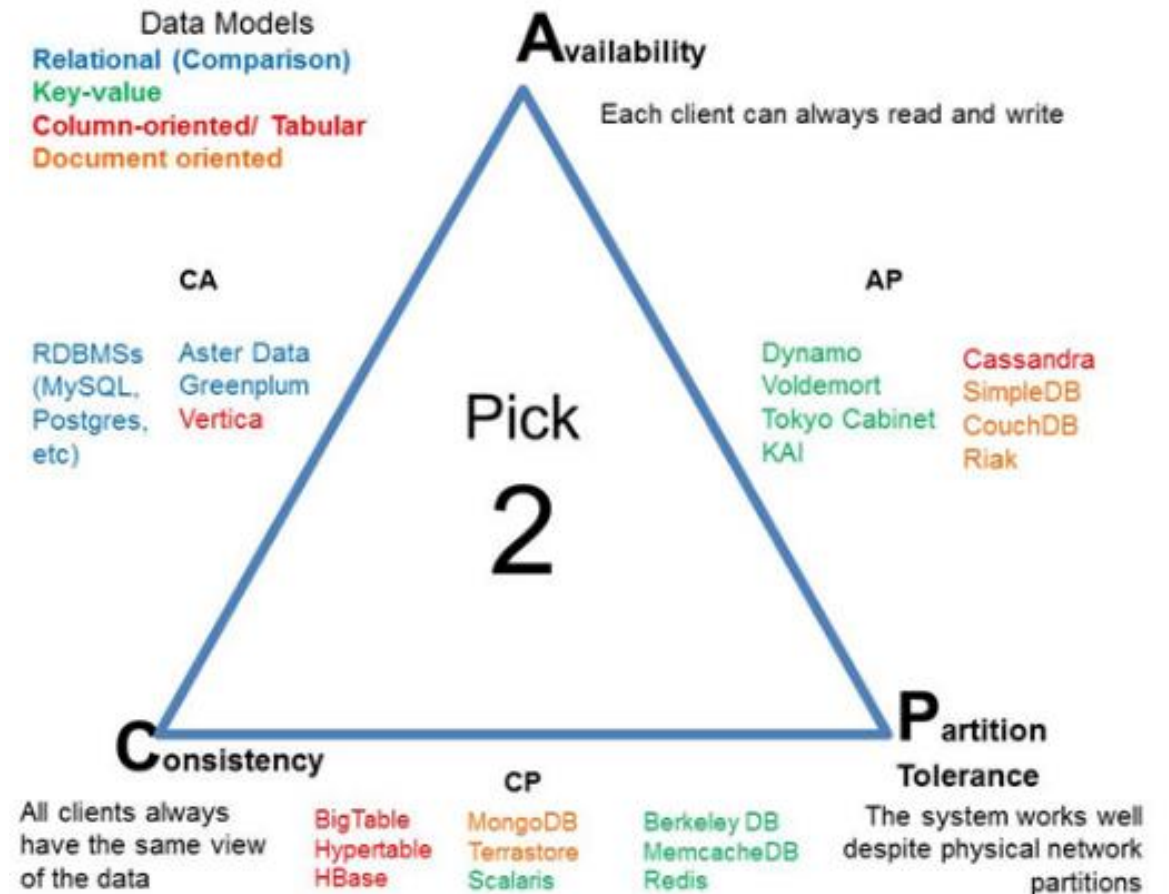
CAP THEOREM

- It is impossible for a distributed computer system to simultaneously provide all three of the following guarantees:
 - **C**onsistency (all nodes see the same data at the same time)
 - **A**vailability (a guarantee that every request receives a response about whether it was successful or failed)
 - **P**artition tolerance (the system continues to operate despite arbitrary message loss or failure of part of the system)

A distributed system can satisfy any two of these guarantees at the same time, but not all three.

CAP THEOREM

- RDBMS are basically CA
 - In some cases it is possible to have a CAP based RDBMS
- I NOSQL systems are mainly CP or AP
 - CP-> data are coherent, but the dbms cannot works 24/7
 - AP -> sometime data cannot be consistent



BASE PRINCIPLE

- Basic Availability: fulfill request, even in partial consistency.
- Soft State: abandon the consistency requirements of the ACID model pretty much completely
- Eventual Consistency: at some point in the future, data will converge to a consistent state; delayed consistency, as opposed to immediate consistency of the ACID properties.
 - purely a liveness guarantee (reads eventually return the requested value)
 - does not make safety guarantees
 - an eventually consistent system can return any value before it converges

A COMPARISON

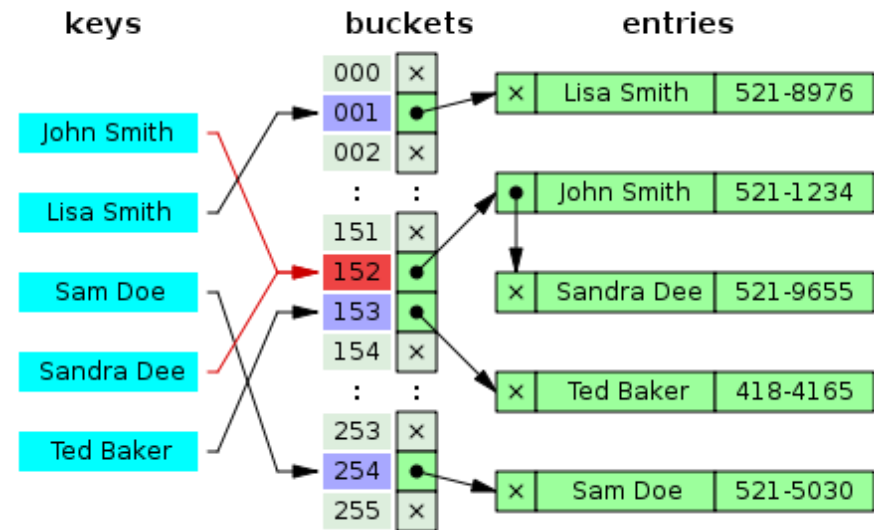
- **ACID (RDBMS):**
 - Strong consistency.
 - Less availability.
 - Pessimistic concurrency.
 - Complex.
- **BASE:**
 - Availability is the most important thing. Willing to sacrifice for this (CAP).
 - Weaker consistency (Eventual).
 - Best effort.
 - Simple and fast.
 - Optimistic

NOSQL MODELS

- Key-Value Stores
- Column Family Stores
- Document Databases
- Graph Databases
- RDF databases as well as Tuple stores (see data semantics course)

KEY VALUE

- Dynamo, Voldemort, Rhino DHT ...
 - DeCandia et al. "Dynamo: Amazon's Highly Available Key-value Store", 2007
- Key-Value store are table where key refers to a given value
- Key-value mapping is based hash algorithm very efficient also wrt. data distribution

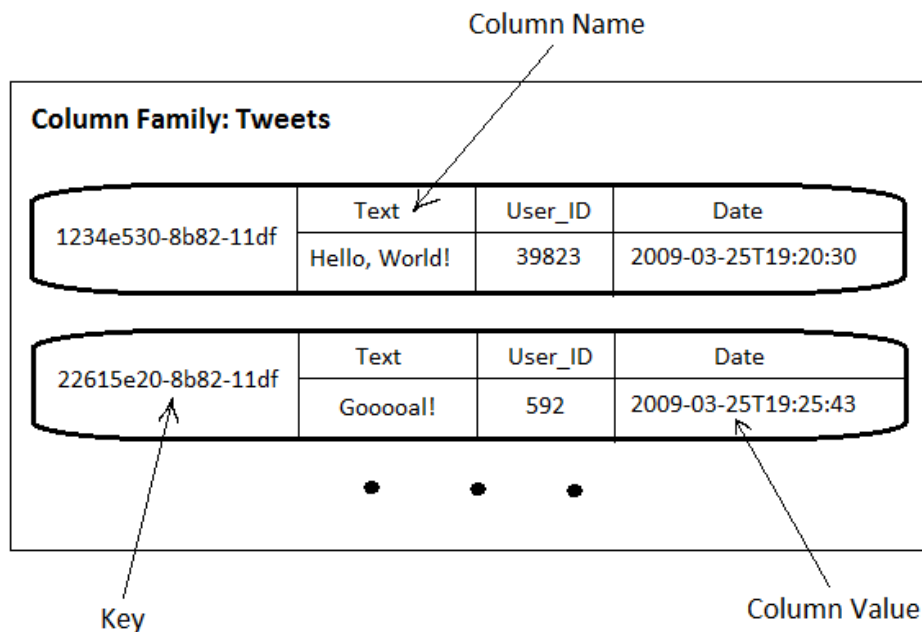


COLUMN

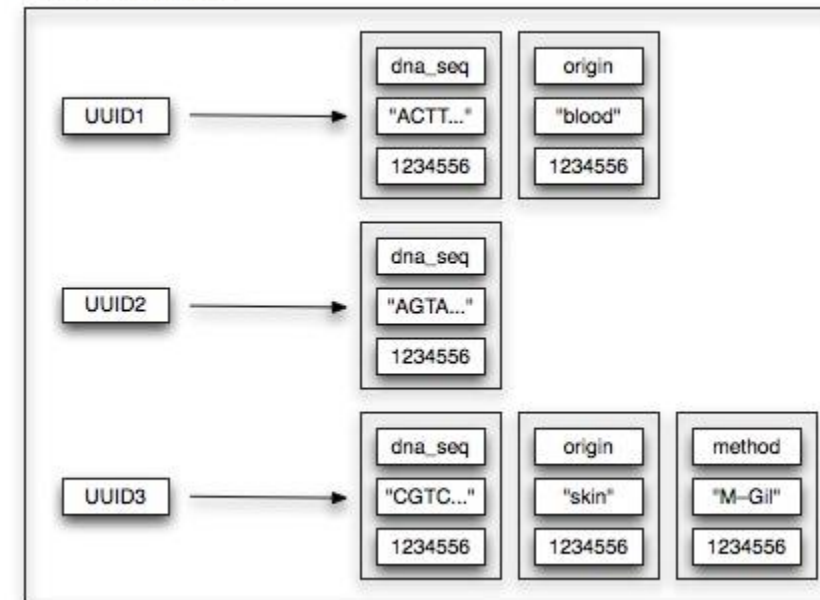
- BigTable, Cassandra, HBase, Hadoop ...
 - Chang et al. "Bigtable: A Distributed Storage System for Structured Data", 2006
- They are able to store big data
 - "Petabytes of data across a large number of servers"
- The key refers to a set of columns

COLUMN FAMILY

jim_87	Age	Name	Gender	Phone
	25	Jim	M	123456
jill_90	Age	Name	Gender	Phone
	22	Jill	F	654321

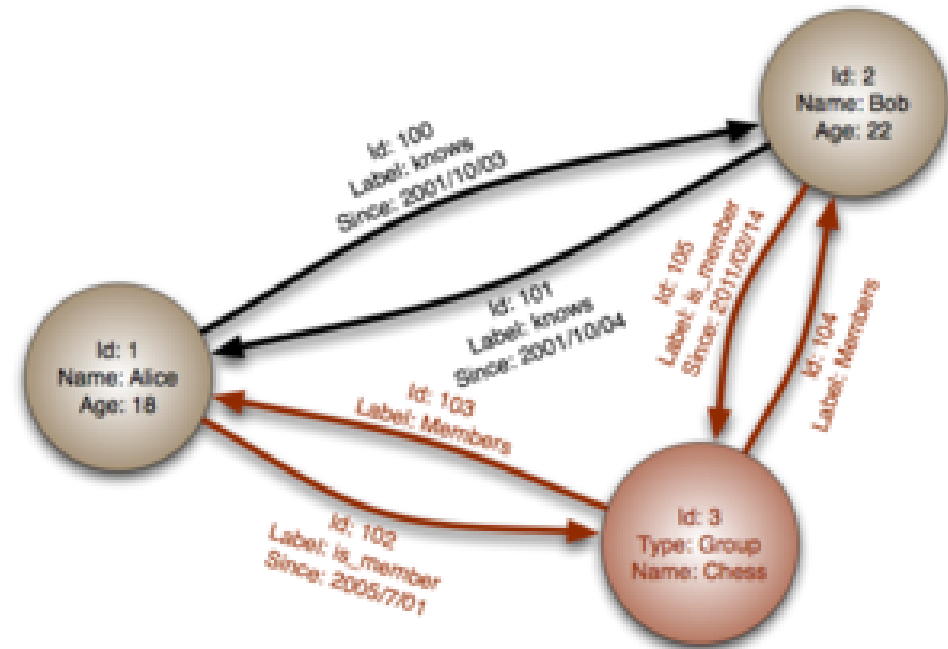


Sequence column family



GRAPH DB

- Neo4J, FlockDB, GraphBase, InfoGrip, ...
- Graphical databases are organized in nodes and the relationships between nodes (edges).
- Nodes and arcs have properties
- Nodes can represent entities (eg "Bob" or "Alice").
- The properties are data related to the node or edge (for example age: 18).
- Graphdbs are not able to scale easily



DOCUMENT BASED

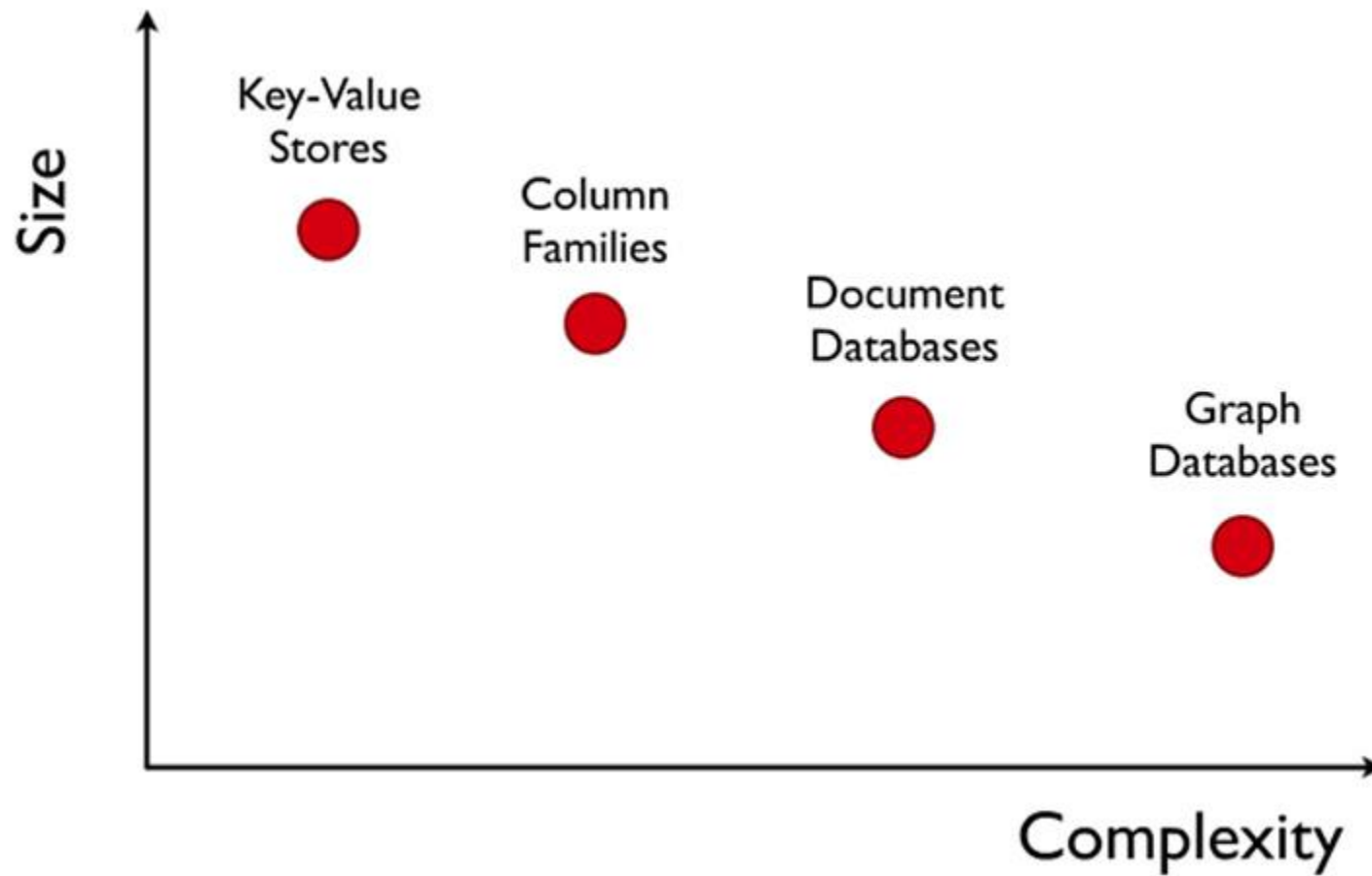
- CouchDB, MongoDB, Lotus Notes, Redis ...
- Documents are addressed in the db by means of unique key (a kind of column family)
- Usually they store data in JSON
- Documents can be searched



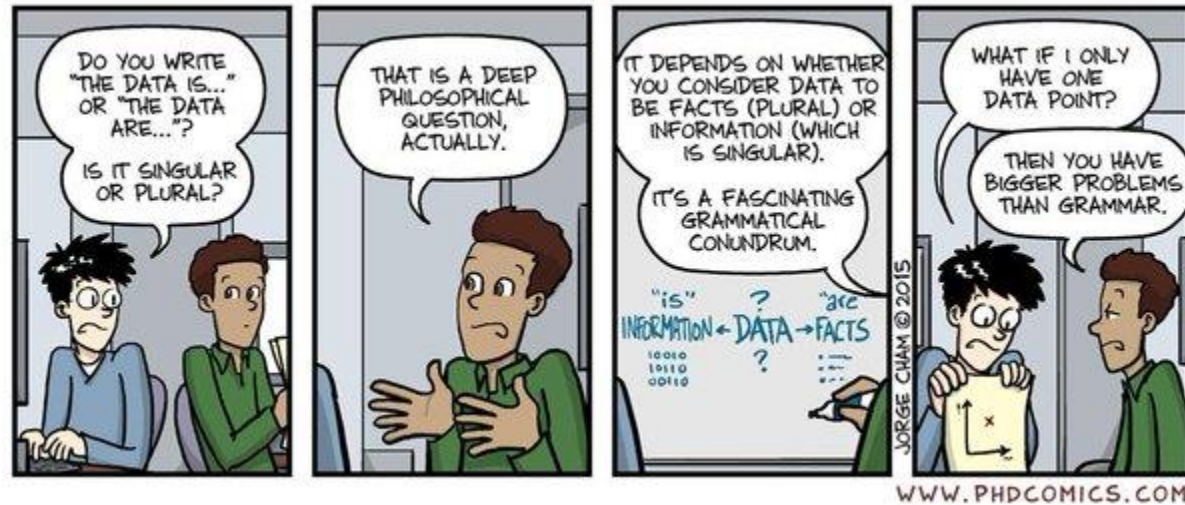
COMPARISON



VOLUME AND COMPLEXITY



DATA IS SINGULAR OR PLURAL?



- In all data models, data connection is a key problem with a graded impact on performance

DATA MODEL

Relational

PERSON			
Pers_ID	Surname	First_Name	City
0	Miller	Paul	London
1	Ortega	Alvaro	Valencia
2	Huber	Urs	Zurich
3	Blanc	Gaston	Paris
4	Bertolini	Fabrizio	Rome

CAR				
Car_ID	Model	Year	Value	Pers_ID
101	Bentley	1973	100000	0
102	Rolls Royce	1965	330000	0
103	Peugeot	1993	500	3
104	Ferrari	2005	150000	4
105	Renault	1998	2000	3
106	Renault	2001	7000	3
107	Smart	1999	2000	2

Document based

```
{
  first_name: 'Paul',
  surname: 'Miller',
  city: 'London',
  location:
[45.123,47.232],
  cars: [
    { model: 'Bentley',
      year: 1973,
      value: 100000, ... },
    { model: 'Rolls Royce',
      year: 1965,
      value : 330000, ... }
  ]
}
```

Relations are included in data

MODEL COMPARISON

ID	Name	Surname	DateofBirth
1	Tom	Hanks	...

Movie	Actor
1	1
2	1
3	1

Join table
Many to many correlation

Id	Title	Director
1	The Da Vinci Code	2
2	The Green Mile	3
3	That thing you do	1
..		



```
{ "Name": "Tom",  
  "Surname": "Hanks",  
  "Works_on": [  
    { "Title": "The Da Vinci Code",  
      "role": "Actor" },  
    { "Title": "That thing you do",  
      "role": ["Actor", "Director"] }  
  ]  
}
```